Algoritmos y Estructuras de Datos II

Laboratorio - 16/04/2026

Laboratorio 3: Tipos de Datos

- Revisión 2026: Franco Luque / Marco Rocchietti

Código

lab03-kickstart.tar.gz

Objetivos

1. Ejercitar la resolución de problemas
2. Uso de arreglos multidimensionales y tipos **`enum`**
3. Uso de arreglos con elementos de tipo **`struct`**
4. Uso de redirección de **`stdout`** por línea de comandos
5. Lectura robusta de archivos

Requerimientos

1. Compilar con los flags de la materia:
   ```
   $ gcc -Wall -Wextra -pedantic -std=c99 …
   ```
2. Seguir las guías de estilo
3. Prohibido usar break, continue y goto!!
4. Prohibido usar return a la mitad de una función.

Recursos

Recursos generales:

- Videos del Laboratorio en el aula virtual
- Documentación en el aula virtual
  - **La biblia: El lenguaje de programación C.**
- Estilo de codificación:
  - Guía de estilo para la programación en C
  - Consejos de Estilo de Programación en C
- Más:
  - Referencia de C (cppreference)
  - The GNU C Library Reference Manual (en inglés)
  - Páginas del manual de linux: Comando “man 3 *<función>*”. Ejemplo: “man 3 scanf”.

Recursos específicos:

- Teóricos:
  - Tipos Concretos
- Prácticos:
  - Práctico 2 - Parte 1
- **Punteros: Comparación del teórico/práctico y el laboratorio**

- Redirección de output
- Descriptores de archivos stdin, stdout y stderr
- **scanf y fscanf:** https://es.cppreference.com/c/io/fscanf
- Punteros: tipos * , desreferenciación o indirección (*), referenciación o dirección (&)
  - 04 - Punteros en C
- Memoria dinámica: sizeof(), malloc(), y free() (también malloc() y realloc())
  - 06 - Memoria Dinámica en C
- Casteo de tipos (type casting)
- Padding en structs:
  - https://www.youtube.com/watch?v=of2kZkUDApw&list=PLLfC2vEod54J7vzFIeX23iyaSAjFhevQV&index=7&t=240s
- Visualizador para C: https://pythontutor.com/c.html#mode=edit
- Violación de segmento

Primera parte: Arreglos Multidimensionales y Estructuras

Ejercicio 1: Datos climáticos

En el directorio del ejercicio se encuentran los siguientes archivos:

Archivo	*Descripción*
`main.c`	Contiene la función principal del programa
`weather.h`	Declaraciones relativas a la estructura de los datos climáticos y de funciones de carga y escritura de datos.
`weather.c`	Implementaciones **incompletas** de las funciones
`weather_table.h`	Declaraciones / prototipos de las funciones que manejan la tabla del clima
`weather_table.c`	Implementaciones **incompletas** de las funciones que manejan la tabla

Parte A: Carga de datos

Abrir el archivo `input/weather_cordoba.in` para ver cómo se estructuran los datos climáticos. Cada línea contiene las mediciones realizadas en un día. Las **primeras tres columnas** corresponden al *año*, *mes* y *día* de las mediciones. Las **restantes seis** columnas son la *temperatura media*, la *máxima*, la *mínima*, la *presión atmosférica*, la *humedad* y las *precipitaciones* medidas ese día.

Las temperaturas se midieron en grados centígrados (ºC) pero para evitar los números reales los grados están expresados en décimas (e.g. 15.2ºC está representado por 152 décimas). La presión (medida en *hectopascales*) también ha sido multiplicada por 10 y las precipitaciones por 100 (o sea que están expresadas en centésimas de milímetro). Esto permite representar todos los datos con números enteros. Cabe aclarar que para completar el ejercicio **no es necesario multiplicar ni dividir estos valores**, esta información es sólo para ayudar a la comprensión de los datos que se manejan.

La primera tarea consiste en completar el procedimiento de carga de datos en el archivo **`weather_table.c`**. También se debe completar **`weather.c`**. Recordar que el programa tiene que ser *robusto*, es decir, debe tener un comportamiento bien definido para los casos en que la entrada no tenga el formato esperado. Como guía se puede revisar el archivo **`array_helpers.c`** provisto por la cátedra en el *laboratorio 1*.

Una vez completada la lectura de datos se puede verificar si la carga funciona compilando:

```
$ gcc -Wall -Wextra -pedantic -std=c99 -c weather_table.c weather.c main.c
$ gcc -Wall -Wextra -pedantic -std=c99 weather_table.o weather.o main.o -o weather
```

y luego ejecutar con el comando:

```
$ ./weather ../input/weather_cordoba.in > weather_cordoba.out
```

En la línea anterior, **`../input/weather córdoba.in`** es el parámetro que se le pasa a nuestro programa **`weather`** (el archivo a procesar) y la parte **`> weather_cordoba.out`** hace que la salida del programa, en vez de mostrarse por la consola, se escriba en el archivo **`weather_cordoba.out.`** El archivo de salida será creado cuando comience la ejecución del programa (si **`weather_cordoba.out`** ya existía va a ser reemplazado).

Si no hubo ningún error, ahora se puede comparar la entrada con la salida:

```
$ diff ../input/weather_cordoba.in weather_cordoba.out
```

El programa **`diff`** (que ya viene instalado en linux) realiza una comparación de ambos archivos y sólo muestra las líneas que difieren. Si esto último no arroja ninguna diferencia, significa que tu carga funciona correctamente.

Parte B: Análisis de los datos

Construir una librería **`weather_utils`** que conste de los siguientes archivos:

- **`weather_utils.c`**
- **`weather_utils.h`**

La librería debe proveer tres funciones:

1. Una función que obtenga la menor temperatura mínima histórica registrada en la ciudad de Córdoba según los datos del arreglo **(práctico 2.1, ejercicio 2.a)**.

2. Un “procedimiento” que registre para cada año entre 1980 y 2016 la mayor temperatura máxima registrada durante ese año **(práctico 2.1, ejercicio 2.b)**.

💡	***El procedimiento debe tomar como parámetro un arreglo que almacenará los resultados obtenidos.***

3. Implementar un procedimiento que registre para cada año entre 1980 y 2016 el mes de ese año en que se registró la mayor cantidad mensual de precipitaciones (campo **`rainfall`**) **(práctico 2.1, ejercicio 2.c)**.
4. Finalmente modificar el archivo **`main.c`** para llamar a todas las funciones e imprimir los resultados obtenidos.

Ayudas:

- Para el procedimiento del *ítem 2* se debería hacer algo parecido a lo siguiente:

```
void procedimiento(WeatherTable a, int output[YEARS]) {
   ⋮
  for (unsigned int year = 0; year < YEARS; year++) {
     ⋮
    output[year] = ... // la mayor temperatura máxima del año  'year' + 1980
     ⋮
  }
}
```

- Para el procedimiento del *ítem 3*:
  - Ver estas filminas: Práctico 2.1: Ejercicio 2
  - Definir una o más funciones auxiliares. Por ejemplo definir **`sum_month_rainfall`**, que dado un año y un mes, devuelve la suma de las precipitaciones de todos los días para ese mes de ese año.
- Para los ítems 2 y 3, chequear que los resultados obtenidos sean iguales a los resultados esperados de la tabla que se muestra a continuación.

Resultados esperados

Año	**ítem 2:** máxima temperatura en un día (en grados * 10)	**ítem 3:** mes de máxima lluvia
1980	380	2
1981	350	1
1982	362	3
1983	384	2
1984	363	1
1985	366	1
1986	397	12
1987	391	12
1988	394	1

1989	386	12
1990	374	1
1991	376	12
1992	337	12
1993	386	1
1994	398	1
1995	408	2
1996	368	11
1997	370	12
1998	380	2
1999	346	12
2000	370	3
2001	363	3
2002	410	1
2003	400	2
2004	393	12
2005	365	1
2006	395	11
2007	374	9
2008	390	2
2009	400	3
2010	394	3
2011	424	2
2012	408	2
2013	400	2
2014	390	2
2015	365	1
2016	380	2

Segunda parte: Punteros 101

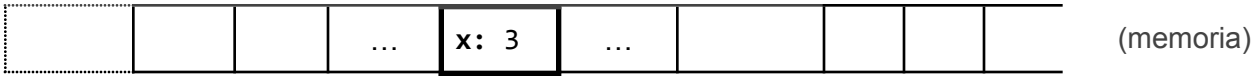
El objetivo del primer ejercicio es adquirir un entrenamiento básico y comprender el funcionamiento de punteros en C.

Un puntero es un tipo de variable especial que guarda una dirección de memoria. En C se denotan los punteros usando el símbolo `*` . Es decir que una variable `p` declarada como `int *p;` es del tipo *puntero a `int`*.

Para el manejo de punteros contamos con dos operadores unarios básicos, el operador de referenciación y el de desreferenciación.

Operación de referenciación (&)

Este operador obtiene la dirección de memoria de una variable. También se lo conoce como operador de dirección (*address operator*). Si se tiene una variable entera **x** declarada como **int x=3;** entonces la expresión **&x** retornará la dirección de memoria donde está alojado el contenido de la variable **x**.

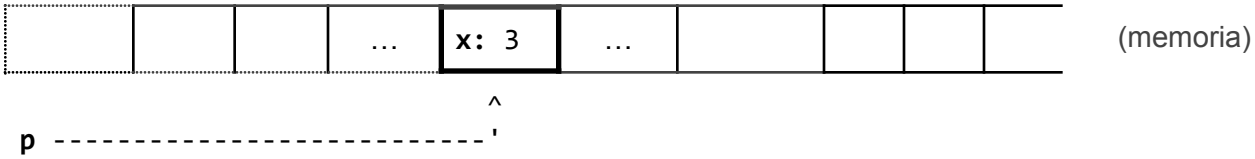


Particularmente la expresión **&x** en este caso es del tipo **int***, es decir del tipo puntero a **int**. Por lo tanto, se puede hacer lo siguiente:

```
int x=3;
int *p;

p = &x;
```

notar que la asignación es correcta porque **p** y **&x** tienen el mismo tipo. En este ejemplo entonces el puntero **p** guarda la dirección de memoria de **x** y podemos decir que ***p*** *apunta a* ***x****.*



Operación de desreferenciación (*)

Obtiene el **valor** de lo *apuntado* por el puntero. También se lo conoce como el operador de indirección (*indirection operator*). Se lo puede pensar como una operación de inspección ya que accede al valor alojado en una dirección de memoria. Si se tiene una variable de tipo **int*** llamada **p**, entonces la expresión ***p** retornará el valor entero que se aloja en la dirección de memoria que contiene **p**. En el ejemplo de más arriba ***p** devuelve **3**.

Además si se utiliza ***p** del lado izquierdo de una asignación:

```
*p = <expresión>;
```

la asignación escribirá el resultado de la expresión en la dirección de memoria apuntada por **p**, por lo que se cambia el valor contenido en esa dirección de memoria (sin modificar a **p**, que seguirá apuntando al mismo lugar). Por ejemplo, si se hace la asignación

```
int x=3;
int *p;

p = &x;
*p = 10;
```

entonces sucede que

Notar que se cambió el valor de la variable **x** de manera indirecta usando el puntero **p**.

Cabe aclarar que cuando se declara la variable de tipo puntero **int *p;** el símbolo ***** no actúa como operador sino que simplemente indica que la variable **p** se declara como puntero.

Para pensar: ¿Qué valor tendrá la variable **y** luego de ejecutar el siguiente código?

```
int x = 3;
int y = 10;
y = *(&x);
```

En el *Laboratorio 1* se utilizaba la función **fscanf()** ¿Qué parámetros tomaba dicha función y cuál era el tipo de cada uno?

Constante nula de punteros (**NULL**)

Siempre es buena idea dar un valor inicial a las variables apenas se declaran. Para punteros existe en C la constante **NULL** que representa una dirección de memoria nula, en la cual no se puede leer ni escribir. Esta constante es una macro definida en los *headers* de **stdlib.h** como la dirección de memoria `0`.

Recordar que si no se inicializa una variable esta puede contener cualquier valor, en el caso de enteros podría ser un número muy grande y extraño (o quizás un inofensivo 4) y en el caso de punteros puede tener asignada una dirección de memoria que en caso de querer escribir o leer de ella generaría problemas. Por ejemplo en el siguiente programa:

```
int *p;
*p = 3;
```

es fácil imaginar que esto podría generar que el programa termine con un error (violación de segmento - *segmentation fault*) pero podría ser peor. Puede suceder que por azar en **p** se encuentre la dirección de memoria de otra variable del programa y la modifiquemos, generando un **BUG** muy difícil de rastrear.

En esta otra versión:

```
int *p=NULL;
*p = 3;
```

el programa siempre va a fallar, y eso es bueno.

Claramente los ejemplos de arriba son errores que saltan a la vista pero sirven para ilustrar situaciones en la que no es tan obvio que se usa un puntero sin inicializar, pero el efecto es el mismo.

Operadores de indexación y flecha

En C además para las variables de tipo puntero se puede usar las operaciones de *indexación* y el *operador flecha* (`->`):

6. Indexación (p**[n]**): Permite obtener el valor que hay en la memoria moviéndose **n** lugares hacia adelante (de manera alineada al tipo de datos del puntero) desde la dirección de memoria guardada en **p**. Entonces por ejemplo **p[0]** es equivalente a ***p**. Cuando se indexa un puntero se debe tener total seguridad de que se va a acceder a memoria asignada a nuestro programa, de lo

contrario ocurrirá un *segmentation fault* (violación de segmento).

7. Acceso indirecto (**->**): Si **p** es un puntero a una estructura **p->member** es un atajo a **(*p).member** (asumiendo que la estructura tiene un campo llamado **member**).

Ejercicio 2: Introducción de punteros

La tarea de este ejercicio consiste en completar el archivo **main.c** de manera tal que la salida del programa por pantalla sea la siguiente:

```
x = 9
m = (100, F)
a[1] = 42
```

Las restricciones son:

- No usar las variables **x**, **m** y **a** en la parte izquierda de alguna asignación.
- Se pueden agregar líneas de código, pero no modificar las que ya existen.
- Se pueden declarar hasta 2 punteros.

Recordar siempre inicializar los punteros en **NULL**.

Ejercicio 3: Simulando parámetros **out** de procedimientos

En el lenguaje de programación del teórico-práctico se usan funciones y procedimientos que tienen una naturaleza distinta a las funciones de C. Particularmente en C sólo existen las funciones y a veces llamamos "procedimientos" a aquellas funciones que devuelven algo del tipo **void** (que es el tipo "vacío" de C, o en otras palabras que no devuelve nada). Se debe entonces traducir el siguiente programa tratando de simular el procedimiento **absolute()** usando funciones de C :

```
proc absolute(in x : int, out y : int)
  if x >= 0 then
    y := x
  else
    y := -x
  fi
end proc

fun main() ret r : int
  var a : int
  a := -10
  absolute(a, res)
  {- supongamos que print() muestra el valor de una variable -}
  print(res)
  {- esta última asignación es análoga a `return EXIT_SUCCESS;` -}
  r := 0
end fun
```

¿Qué valor se mostraría al ejecutar la función **`main()`** del programa anterior?

a) Abrir el archivo **`abs1.c`** y traducir a lenguaje C usando el siguiente prototipo para **`absolute()`**:

```
void absolute(int x, int y) {
  (⋮)
}
```

luego compilar con el siguiente comando:

```
$ gcc -Wall -Werror -pedantic -std=c99 abs1.c -o abs1
```

(notar que no se utiliza **`-Wextra`** sólo por esta vez) y ejecutar

```
$ ./abs1
```

¿Qué valor se muestra por pantalla? ¿Coincide con el programa en el lenguaje del teórico? Responder en un comentario al final de **`abs1.c`**.

b) Ahora abrir el archivo **`abs2.c`** que utiliza el siguiente prototipo de **`absolute()`**:

```
void absolute(int x, int *y) {
  (⋮)
}
```

Pensar qué modificaciones son necesarias hacer dentro de las funciones **`absolute()`** y **`main()`** respecto a la implementación realizada en **`abs1.c`** para trabajar con el nuevo prototipo. Además se debe lograr que el programa en C simule el comportamiento del programa original en lenguaje del teórico-práctico. Implementar esas modificaciones en **`abs2.c`** y luego compilar:

```
$ gcc -Wall -Werror -Wextra -pedantic -std=c99 abs2.c -o abs2
```

y por último ejecutar

```
$ ./abs2
```

¿Se muestra el valor correcto? (en caso contrario revisar hasta lograr que sí lo haga)

❗	*Notar que conceptualmente en ningún caso los programas son equivalentes puesto que las funciones y procedimientos del lenguaje del teórico tienen una naturaleza completamente distinta a las de C.*

c) Para pensar:

- ¿El parámetro **`int *y`** de `absolute()` es de tipo **`in`**, de tipo **`out`** o de tipo **`in/out`**?
- ¿Qué tipo de parámetros tiene disponibles C para sus funciones?
  - Parámetros **`in`**
  - Parámetros **`out`**
  - Parámetros **`in/out`**

Responder al final de **abs2.c** como un comentario en el código

d) En un nuevo archivo **swap.c** implementar una traducción del programa que intercambia valores:

```
proc swap(in/out a : int, in/out b : int)
    var aux : int
    aux := a
    a := b
    b := aux
end proc

fun main() ret r : int
  var x, y : int
  x := 6
  y := 4
  print(x, y)
  swap(x, y)
  print(x, y)
  r := 0
end fun
```

Tercera parte: Memoria dinámica

Visualizando direcciones de memoria

Los valores de las variables del tipo puntero (las direcciones de memoria) se pueden visualizar. Por lo general esto no se hace, salvo a veces para hacer *debug*. La manera es usando **printf()** con **%p**:

```
int *p=NULL;
int a=55;
p = &a;
printf("La dirección de memoria apuntada por p es: %p", (void *) p);
```

Notar el *casteo* que se le realiza a **p** cuando se lo pasa como parámetro a **printf()**. Cuando antes de una expresión se introduce un tipo entre paréntesis, significa que la expresión se va a convertir al tipo indicado. Por ejemplo **(float) 2** hace que el resultado se interprete como **2.0f**, otro ejemplo puede ser **(int) 1.5** que hace que el resultado sea el entero **1**. En este caso se convierte a **p**, que es un **int***, a un **void*** o sea un puntero a **void** lo cual es simplemente una dirección de memoria pura, puesto que un puntero a **void** no se puede desreferenciar.

La salida del programa va a ser un número en hexadecimal (con prefijo `0x`…), por ejemplo:

```
La dirección de memoria apuntada por p es: 0x7ffd15a1ac60
```

Otra forma con la cual se puede ver el valor decimal de una dirección de memoria es hacer:

```
int *p=NULL;
int a=55;
```

```
p = &a;
printf("El índice de memoria alojado en p es: %lu", (uintptr_t) p);
```

aquí se castea el puntero a un entero sin signo que es el tipo **`uintptr_t`** definido en **`<stdint.h>`**. Este tipo es lo suficientemente grande para guardar direcciones de memoria, pero la solución no es muy estándar ya que el comodín **`"%lu"`** podría fallar en algunos sistemas ya que **`printf()`** espera algo del tipo **`long unsigned int`** y podría no ser equivalente a **`uintptr_t`**. La salida sería algo como:

```
El índice de memoria alojado en p es: 140724966370400
```

Memoria dinámica: Stack vs Heap

En el lenguaje del teórico práctico se usa el procedimiento **`alloc()`** para reservar memoria para un puntero, y **`free()`** para liberar dicha memoria:

```
var p: pointer to int

alloc(p)
*p := 5
free(p)
```

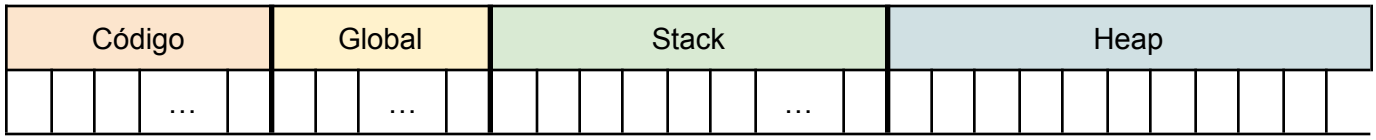
En C esto se hace usando las funciones **`malloc()`** y **`free()`**:

```
int *p=NULL;
p = malloc(sizeof(int));
*p = 5;
free(p);
```

La función **`malloc()`** toma un parámetro, que es un entero sin signo de tipo **`size_t`** (muy parecido a **`unsigned long int`**) que es la cantidad de memoria en *bytes* que se solicita reservar. A diferencia de **`alloc()`** del teórico, que automáticamente reserva la cantidad necesaria según el tipo de puntero, en C hay que indicar explícitamente la cantidad de *bytes* a reservar. El operador **`sizeof()`** devuelve la cantidad de *bytes* ocupados por una expresión o tipo, por lo que resulta indispensable para el uso de **`malloc()`** (aún si uno hubiera memorizado cuantos *bytes* ocupa cada tipo en su computadora, esto puede variar según la versión del sistema operativo o el microprocesador en el que se use el programa).

```
$ man malloc
```

Las direcciones de memoria que devuelve **`malloc()`** se encuentran en la sección de memoria denominada *Heap* (no confundir con la estructura de datos que lleva el mismo nombre ya que no tiene ninguna relación).



A modo informativo, el mapa de más arriba es un esquema de cómo se organiza la memoria. La sección de *Código* contiene las instrucciones del programa, la sección *Global* contiene las variables globales, la sección *Stack* es donde están las variables que usamos en las funciones de nuestro programa (memoria estática) y la sección *Heap* es la región de la memoria dinámica la cual se reserva y libera manualmente mediante **`malloc()`** y **`free()`**. El *Stack* por su parte se maneja de manera automática, reservando memoria para las variables declaradas en una función que se comienza a ejecutar y liberando esa memoria cuando la función termina su ejecución.

Otra gran diferencia entre el *Stack* y el *Heap*, es que la cantidad de memoria asignada para el Stack es limitada. Si los datos contenidos en el *Stack* superan dicho límite se genera un ***stack overflow***. Si durante la ejecución de un programa se está ejecutando una función **`f1`** y ésta llama a su vez a otra función **`f2`**, durante la ejecución de **`f2`** las variables de **`f1`** siguen en el *Stack* ya que aún no se terminó de ejecutar. Las variables de las funciones llamadas de manera anidada se van apilando entonces. Hay en consecuencia un límite en la cantidad de llamadas anidadas de funciones, particularmente un número máximo de llamadas recursivas. La cantidad dependerá de cuánta memoria ocupen las variables de las funciones involucradas. Esto hace que si una función declara un arreglo en memoria estática muy grande, podría dejar poco margen para llamadas a otras funciones o directamente generar un *stack overflow* porque el arreglo no entra en el *Stack*.

Por su parte la memoria en el *Heap* tiene disponible toda la memoria *RAM* de la computadora, por lo que mientras haya memoria libre se podrá pedir reservar nueva memoria mediante **`malloc()`**. Pero *un gran poder conlleva una gran responsabilidad*, por lo que no se debe olvidar liberar la memoria reservada cuando deje de usarse, puesto que los ***memory leaks*** pueden generar a la larga que la computadora se bloquee por completo.

Ejercicio 4: `sizeof`, `malloc` y `free`

a)

1. Completar el archivo **`sizes.c`** para que muestre el tamaño en *bytes* de cada miembro de la estructura **`data_t`** por separado y el <u>tamaño total</u> que ocupa la estructura en memoria. *¿La suma de los miembros coincide con el total? ¿El tamaño del campo* **`name`** *depende del nombre que contiene?*
2. De manera similar a lo hecho para mostrar los tamaños de cada campo de la estructura, agregar un mensaje que muestre la dirección de memoria de cada campo. Se recomienda usar los dos tipos de visualizaciones explicadas anteriormente (direcciones e índices). Analizar la salida y responder: *¿Hay algo raro en las direcciones de memoria?*
   Ayuda: Ver recursos sobre padding de estructuras al principio del documento.

3. Por último, modificar **`sizes.c`** para que la estructura `data_t` se guarde en la memoria dinámica. Usar **`malloc`** y **`free`** para reservar y liberar la memoria.
   Para guardar el campo name, se puede usar la función **`strcpy`** de **`string.h`**.

b) En el directorio **`static`** se encuentra el programa del Laboratorio 1 que carga en un arreglo en memoria estática desde un archivo. Completar en la carpeta **`dynamic`** la función **`array_from_file()`** de **`array_helpers.c`**:

```
int *array_from_file(const char *filepath, size_t *length);
```

que carga los datos del archivo **`filepath`** devolviendo un puntero a memoria dinámica con los elementos del arreglo y dejando en **`*length`** la cantidad de elementos leídos. Completar además en **`main.c`** el código necesario para liberar la memoria utilizada por el arreglo. Probar el programa con todos los archivos de la carpeta **`input`** para asegurar el correcto funcionamiento (notar que la versión en **`static`** no funciona para todos los archivos de la carpeta **`input)`**.